UNIT- III

Dynamic programming : general method – multi stage graphs – all pairs shortest path algorithm – 0/1 Knapsack and traveling salesman problem – chained matrix multiplication – approaches using recursion – memory functions. **Basic search and traversal technique :** the technique – and/or graphs – biconnected components – depth first search – topological sorting – breadth first search.

3.1. DYNAMIC PROGRAMMING

3.1.1. THE GENERAL METHOD

- It is an algorithm design method that can be used when the solution to a problem can be viewed as the result of a sequence of decisions.
- The idea of dynamic programming is thus quit simple: avoid calculating the same thing twice, usually by keeping a table of known result that fills up a sub instances are solved.
- Divide and conquer is a top-down method. When a problem is solved by divide and conquer, we immediately attack the complete instance, which we then divide into smaller and smaller sub-instances as the algorithm progresses.
- Dynamic programming on the other hand is a bottom-up technique. We usually start with the smallest and hence the simplest sub- instances. By combining their solutions, we obtain the answers to sub-instances of increasing size, until finally we arrive at the solution of the original instances.
- The essential difference between the greedy method and dynamic programming is that the greedy method only one decision sequence is ever generated. In dynamic programming, many decision sequences may be generated. However, sequences containing sub-optimal sub-sequences can not be optimal and so will not be generated.
- Because of principle of optimality,decision sequences containing subsequences that are suboptimal are not considered. Although the total number of different descision sequences is exponential in the number of descisions(if there are d choices for each of the n descisions to be made then there are d^n possible decision sequences),Dynamic programming algorithms often have a polynomial complexity.

3.1.2. MULTISTAGE GRAPH

1. A multistage graph G = (V,E) is a directed graph in which the vertices are portioned into k > = 2 disjoint sets V_i, 1 <= i<= k.
2. In addition, if < u,v > is an edge in E, then u < = V_i and V $\sum V_{i+1}$ for some i, 1<= i < k.
3. If there will be only one vertex, then the sets V_i and V_k are such that $[V_i]=[V_k] = 1$.
4. Let ‘s’ and ‘t’ be the source and destination respectively.

5. The cost of a path from source (s) to destination (t) is the sum of the costs of the edger on the path.
6. The multistage graph problem is to find a minimum cost path from 's' to 't'.
7. Each set V_i defines a stage in the graph. Every path from 's' to 't' starts in stage-1, goes to stage-2 then to stage-3, then to stage-4, and so on, and terminates in stage-k.

8. This Mulistage Graph problem can be solved in 2 ways.
   a) Forward Method.
   b) Backward Method.

3.1.2.1. FORWARD METHOD

Assume that there are 'k' stages in a graph. In this Forward approach, we will find out the minimum cost path starting from the 1st stage to k^{th} stage

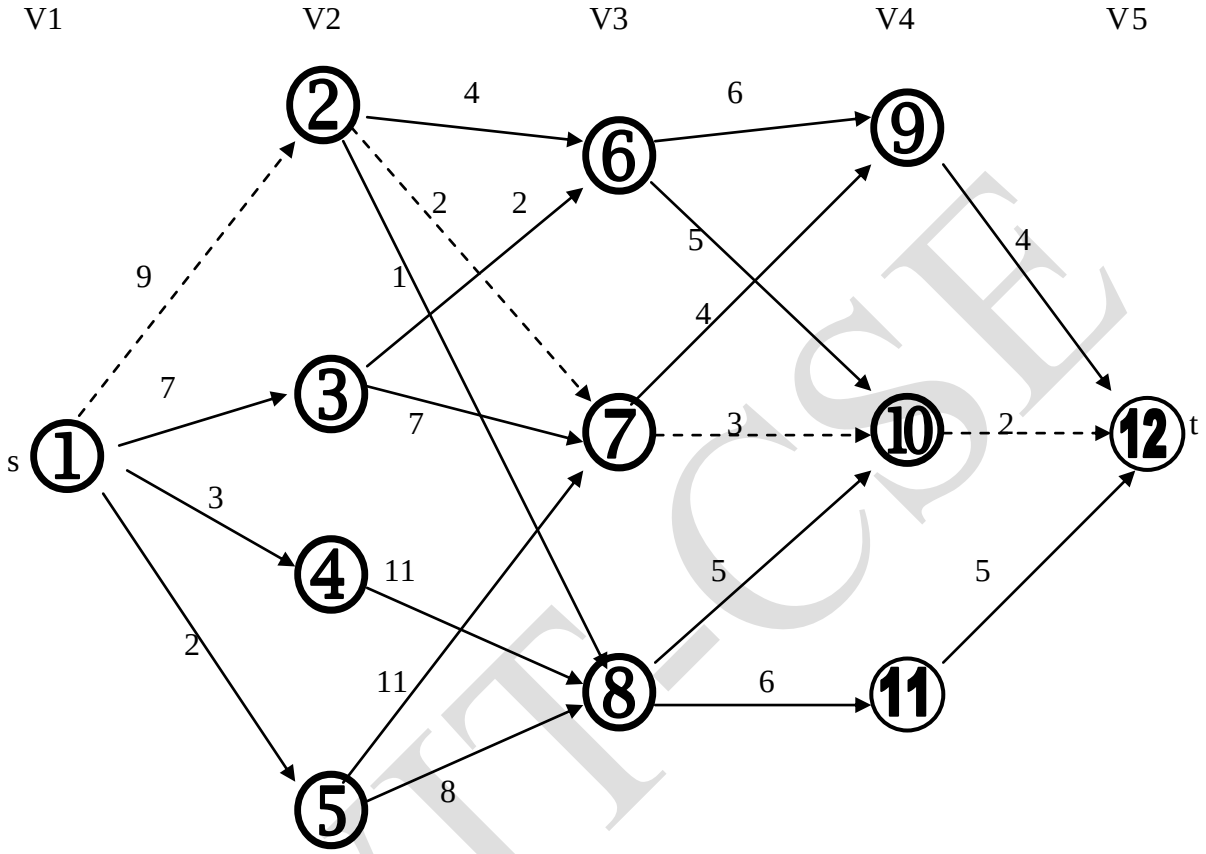
Procedure:

- Maintain a cost matrix cost(n) which stores the distance from any vertex to the destination.
- If a vertex is having more than one path, then we have to choose the minimum distance path and the intermediate vertex which gives the minimum distance path will be stored in the distance array 'D'.
- In this way we will find out the minimum cost path from each and every vertex.
- Finally cost(1) will give the shortest distance from source to destination.
- For finding the path, start from vertex-1 then the distance array D(1) will give the minimum cost neighbour vertex which in turn give the next nearest vertex and proceed in this way till we reach the Destination.
- For a 'k' stage graph, there will be 'k' vertex in the path.
- In the above graph $V_1 \ldots\ldots V_5$ represent the stages. This 5 stage graph can be solved by using forward approach as follows,

Algorithm:

```
Algorithm FGraph (G,k,n,p)
//  The input is a k-stage graph G=(V,E) with 'n' vertex  indexed in order of stages.
//  E is a set of edges.
//  c[i,j] is the cost of <i,j>
//  p[1:k] is a minimum cost path.
{
    cost[n]=0.0;
    for j=n-1 to 1 step-1 do
    {
       //compute cost[j],
         Let 'r' be the vertex such that <j,r> is an edge of 'G' &
     c[j,r]+cost[r] is minimum.
         cost[j] = c[j+r] + cost[r];
         d[j] =r;
    }
    // find a minimum cost path.
    p[1]=1;
    p[k]=n;
    for j=2 to k-1 do
        p[j]=d[p[j-1]];
}
```

MVIT-CSE



TRACE OUT:

n=12 k=5

cost (12)=0 d (12)=0

for j = 11 to 1

When j=11

cost (11)=5 d (11)=12

When j=10

cost (10)=2 d (10)=12

When j=9

cost (9)=4 d (9)=12

Note:

If there is more than one path from vertex j to next stage vertices,then we find the closest vertex by the following formula:

- For forward approach,

$$\text{Cost}(i,j) = \min_{\substack{l \in V_{i+1} \\ (j,l) \in E}} \{C(j,l) + \text{Cost}(i+1,l)\}$$

When j=8

cost(8) = min {C (8,10) + Cost (10), C (8,11) + Cost (11) }

MVIT-CSE

= min (5 + 2, 6 + 5)
= min (7,11)
= 7
cost(8) =7 d(8)=10

When j=7
cost(7) = min(c (7,9)+ cost(9),c (7,10)+ cost(10))
= min(4+4,3+2)
= min(8,5)
= 5
cost(7) = 5 d(7) = 10

When j=6
cost(6) = min (c (6,9) + cost(9),c (6,10) +cost(10))
= min(6+4 , 5 +2)
= min(10,7)
= 7
cost(6) = 7 d(6) = 10

When j=5
cost(5) = min (c (5,7) + cost(7),c (5,8) +cost(8))
= min(11+5 , 8 +7)
= min(16,15)
= 15
cost(5) = 15 d(5) = 18

When j=4
cost(4) = min (c (4,8) + cost(8))
= min(11+7)
= 18
cost(4) = 18 d(4) = 8

When j=3
cost(3) = min (c (3,6) + cost(6),c (3,7) +cost(7))
= min(2+7 , 7 +5)
= min(9,12)
= 9
cost(3) = 9 d(3) = 6

When j=2
cost(2) = min (c (2,6) + cost(6),c (2,7) +cost(7) ,c (2,8) +cost(8))
= min(4+7 , 2+5 , 1+7)
= min(11,7,8)
= 7
cost(2) = 7 d(2) = 7

When j=1
cost(1)= min (c (1,2)+cost(2) ,c (1,3)+cost(3) ,c (1,4)+cost(4) c(1,5)+cost(5))

= min(9+7 , 7 +9 , 3+18 , 2+15)
= min(16,16,21,17)
= 16
cost(1) = 16 d(1) = 2

p[1]=1 p[5]=12
for j=2 to 4
When j=2 p[2]=d[p[1]]=d[1]=2

MVIT-CSE

When j=3 p[3]=d[p[2]]=d[2]=7
When j=4 p[4]=d[p[3]]=d[7]=10

The path through which you have to find the shortest distance from vertex 1 to vertex 12.

(ie) $\rightarrow$ ① $\rightarrow$ ② $\rightarrow$ ⑦ $\rightarrow$ ⑩ $\rightarrow$ ⑫

p[1] p[2] p[3] p[4] p[5]

Start from vertex - 1

D (1) = 2
D (2) = 7
D (7) = 10
D (10) = 12

So, the minimum –cost path is,

① $\xrightarrow{9}$ ② $\rightarrow$ ⑦ $\rightarrow$ ⑩ $\rightarrow$ ⑫

∴ The cost is 9+2+3+2+=16

ANALYSIS:

The time complexity of this forward method is $O(|V|+|E|)$

3.1.2.2. BACKWARD METHOD

- If there are 'k' stages in a graph then using backward approach, we will find out the minimum cost path from destination to source (ie)[from stage k to stage 1]

Procedure:

1. It is similar to forward approach, but differs only in two or three ways.
2. Maintain a cost matrix to store the cost of every vertices and a distance matrix to store the minimum distance vertex.
3. Find out the cost of each and every vertex starting from vertex 1 up to vertex k.
4. To find out the path star from vertex 'k', then the distance array D (k) will give the minimum cost neighbor vertex which in turn gives the next nearest neighbor vertex and proceed till we reach the destination.

Algorithm :

```
Algorithm BGraph (G,k,n,p)
// The i/p is a k-stage graph G=(V,E) with 'n' vertex indexed in order of stages
// E is a set of edges.
// c[i,j] is the cost of<i,j>
// p[1:k] is a minimum cost path.
{
    bcost[1]=0.0;
     for j=2 to n do
    {
            //compute bcost[j],
           Let 'r' be the vertex such that <r,j> is an edge of 'G' & bcost[r]+c[r,j] is
```


MVIT-CSE

```
            minimum.
            bcost[j] = bcost[r] + c[r,j];0
           d[j] =r;
    }
  // find a minimum cost path.
  p[1]=1;
  p[k]=n;
  for j= k-1 to 2 do
        p[j]=d[p[j+1]];
}
```

TRACE OUT:

```
n=12      k=5
  bcost (1)=0                    d (1)=0
for  j = 2 to 12
When j=2
        bcost(2) = 9                d(2)=1
When j=3
        bcost(3) = 7                d(3)=1
When j=4
        bcost(4) = 3                d(4)=1
When j=5
        bcost(5) = 2                d(5)=1
When j=6
        bcost(6) =min(c (2,6) + bcost(2),c (3,6) + bcost(3))
                =min(13,9)
        bcost(6)  = 9               d(6)=3
When j=7
        bcost(7) =min(c (3,7) + bcost(3),c (5,7) + bcost(5) ,c (2,7) + bcost(2))
                =min(14,13,11)
        bcost(7)  = 11              d(7)=2
When j=8
        bcost(8) =min(c (2,8) + bcost(2),c (4,8) + bcost(4) ,c (5,8) +bcost(5))
                =min(10,14,10)
        bcost(8)  = 10              d(8)=2
When j=9
        bcost(9) =min(c (6,9) + bcost(6),c (7,9) + bcost(7))
                =min(15,15)
        bcost(9)  = 15              d(9)=6
When j=10
        bcost(10)=min(c(6,10)+bcost(6),c(7,10)+bcost(7)),c(8,10)+bcost(8))
                =min(14,14,15)
        bcost(10)= 14               d(10)=6
When j=11
        bcost(11) =c (8,11) + bcost(8))
        bcost(11) = 16              d(11)=8
When j=12
        bcost(12)=min(c(9,12)+bcost(9),c(10,12)+bcost(10),c(11,12)+bcost(11))
                =min(19,16,21)
        bcost(12) = 16              d(12)=10
```

MVIT-CSE

p[1]=1 p[5]=12
for j=4 to 2
When j=4 p[4]=d[p[5]]=d[12]=10
When j=3 p[3]=d[p[4]]=d[10]=6
When j=2 p[2]=d[p[3]]=d[6]=3

The path through which you have to find the shortest distance from vertex 1 to vertex 12.

p[5]=12
p[4]= =10
p[3]= =6
p[2]= =3
p[1]=1

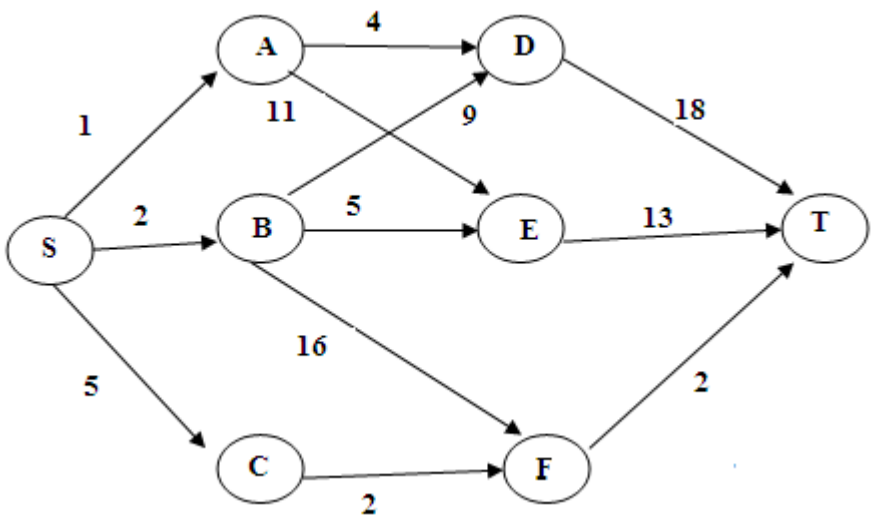
So the minimum cost path is,

$$1 \xrightarrow{7} 3 \xrightarrow{2} 6 \xrightarrow{5} 10 \xrightarrow{2} 12$$

The cost is 16.

Solve problems for practice:

1. **Solve the multistage graph using dynamic programming (Source is S and T is the destination. Both the approaches)**



3.1.3. ALL PAIRS SHORTEST PATH ALGORITHM

- Let G = <V,E> be a directed graph with n vertices , 'E' is the set of edges & V is the set of n vetices.
- Each edge has an associated non-negative length.
- We want to calculate the length of the shortest path between each pair of nodes.

i.e)Shortest path between every vertex to all other vertices.

- Suppose the nodes of G are numbered from 1 to n, so n={1,2,...n}
- cost (i,j) is length of edge<i,j> and it is called as cost adjacency matrix.
  - cost(i,i) = 0 for i=1,2...n,
  - cost(i,j) = cost of edge <i,j> for all i&j
  - cost (i,j) = infinity, if the edge (i,j) does not exist.

- The principle of optimality applies:if k is the node on the shortest path from i to j then the part of the path from i to k and the part from k to j must also be optimal, that is shorter.

Algorithm:

```
Algorithm Allpaths(cost,A,n)
//cost[1…n,1…n] is the cost adjacency matrix of graph with n vertices
//A[i,j] is the cost of shortest path from vertex i to j
//cost[I,I]=0.0 for 1<=i<=n
{
    for i:=1 to n do
        for j:=1 to n do
            A[i,j]:=cost[i,j]; //copy cost into A
        for k:=1 to n do
        for i:=1 to n do
            for j:=01 to n do
                A[i,j]:=min{A[i,j], A[i,k]+ A[k,j];
}
```

- First create a cost adjacency matrix for the given graph.
- Copy the above matrix to matrix D,which will give the direct distance between nodes.
- We have to perform n iterations after iteration k. The matrix D will give you the distance between nodes with only (1,2...,k)as intermediate nodes.
- At the iteration k,we have to check for each pair of nodes(i,j) whether or not there exists a path from i to j passing through node k.
- Likewise we have to find the value for n iterations (ie) for n nodes.

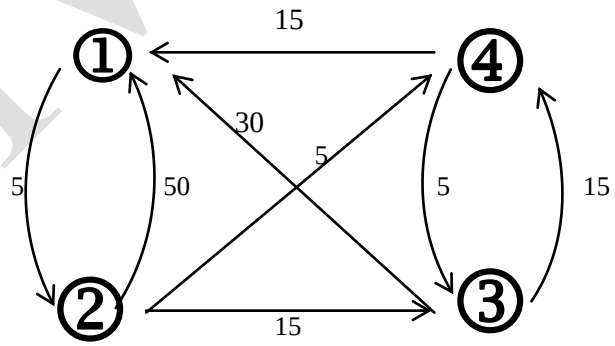


Fig: floyd's algorithm and work

Cost Adjacency Matrix:

❖ At 0th iteration it nil give you the direct distances between any 2 nodes

$$\text{cost}(i,j) = A(i,j) = D_0 = \begin{vmatrix} 0 & 5 & \propto & \propto \\ 50 & 0 & 15 & 5 \\ 30 & \propto & 0 & 15 \\ 15 & \propto & 5 & 0 \end{vmatrix}$$

MVIT-CSE

It gives the direct distances between the two nodes.

- At 1st iteration we have to check the each pair(i,j) whether there is a path through node 1.if so we have to check whether it is minimum than the previous value and if i is so than the distance throug0h 1 is the value of $d_1(i,j)$.at the same time we have to solve the intermediate node in the matrix position p(i,j).

for k= 1 to 4
when k=1
for i= 1 to 4
when i=1
for j= 1 to 4
when j=1
A[1,1]=min(A[1,1],a[1,1]+A[1,1])=min(0,0)=0
when j=2
A[1,2]=min(A[1,2],A[1,1]+A[1,2])=min(5,0+5)=5
when j=3
A[1,3]=min(A[1,3],A[1,1]+A[1,3])=min(∝,0+∝)=∝
when j=4
A[1,4]=min(A[1,4],A[1,1]+A[1,4])=min(∝,0+∝)=∝

when i=2
for j= 1 to 4
when j=1
A[2,1]=min(A[2,1],A[2,1]+A[1,1])=min(50,50+0)=50
When j=2
A[2,2]=min(A[2,2],A[2,1]+A[1,2])=min(0,50+5)=0
When j=3
A[2,3]=min(A[2,3],A[2,1]+A[1,3])=min(15,50+∝)=15
When j=4
A[2,4]=min(A[2,4],A[2,1]+A[1,4])=min(5,50+∝)=5

When i=3
for j= 1 to 4
when j=1
A[3,1]=min(A[3,1],A[3,1]+A[1,1])=min(30,30+0)=30
When j=2
A[3,2]=min(A[3,2],A[3,1]+A[1,2])=min(∝,30+5)=35
When j=3
A[3,3]=min(A[3,3],A[3,1]+A[1,3])=min(0,30+∝)=0
When j=4
A[3,4]=min(A[3,4],A[3,1]+A[1,4])=min(15,30+∝)=15

When i=4
for j= 1 to 4
when j=1
A[4,1]=min(A[4,1],A[4,1]+A[1,1])=min(15,15+0)=15
When j=2
A[4,2]=min(A[4,2],A[4,1]+A[1,2])=min(∝,15+5)=20
When j=3
A[4,3]=min(A[4,3],A[4,1]+A[1,3])=min(5,15+∝)=5

MVIT-CSE

When j=4

A[4,4]=min(A[4,4],A[4,1]+A[1,4])=min(0,15+∝)=0

	i	j	A[i,j]
1	1	1	0
		2	5
		3	∝
		4	∝
	2	1	50
		2	0
		3	15
		4	5
	3	1	30
		2	**35**
		3	0
		4	15
	4	1	15
		2	**20**
		3	5
		4	0

$$D_1=\begin{vmatrix} 0 & 5 & \propto & \propto \\ 50 & 0 & 15 & 5 \\ 30 & \mathbf{35} & 0 & 15 \\ 15 & \mathbf{20} & 5 & 0 \end{vmatrix} \quad \begin{matrix} P[3,2]=1 \\ P[4,2]=1 \end{matrix} \quad P_1=\begin{vmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{vmatrix}$$

When k=2,we have obtained the matrix D_2 and P_2

$$D_2=\begin{vmatrix} 0 & 5 & \mathbf{20} & \mathbf{10} \\ 50 & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{vmatrix} \quad \begin{matrix} P[1,3]=2 \\ P[1,4]=2 \end{matrix} \quad P_2=\begin{vmatrix} 0 & 0 & 2 & 2 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{vmatrix}$$

$$D_3=\begin{vmatrix} 0 & 5 & 20 & 10 \\ \mathbf{45} & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 5 & 20 & 5 & 0 \end{vmatrix} \quad P[2,1]=3 \quad P_3=\begin{vmatrix} 0 & 0 & 2 & 2 \\ 3 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{vmatrix}$$

$$D4=\begin{vmatrix} 0 & 5 & \mathbf{15} & 10 \\ 20 & 0 & \mathbf{10} & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{vmatrix} \quad \begin{matrix} P[1,3]=4 \\ P[2,3]=4 \end{matrix} \quad P4=\begin{vmatrix} 0 & 0 & 4 & 2 \\ 3 & 0 & 4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{vmatrix}$$

MVIT-CSE

- D_4 will give the shortest distance between any pair of nodes.
- If you want the exact path then we have to refer the matrix p

- The matrix will be,

$$P = \begin{vmatrix} 0 & 0 & 4 & 2 \\ 3 & 0 & 4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{vmatrix} \qquad 0 \longrightarrow \text{direct path}$$

-
- Since,p[1,3]=4,the shortest path from 1 to3 passes through 4.
- Looking now at p[1,4]&p[4,3] we discover that between 1 & 4, we have to go to node 2 but that from 4 to 3 we proceed directly.
- Finally we see the trips from 1 to 2, & from 2 to 4, are also direct.
- The shortest path from 1 to 3 is 1,2,4,3.

Algorithm:

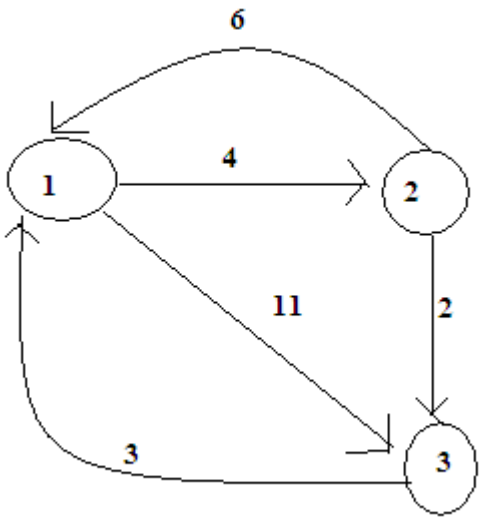
```
Function Floyd (L[1..r,1..r]):array[1..n,1..n]
                array D[1..n,1..n]
D = L
        for   k = 1 to n do
                for   i  = 1 to n do
                        for   j  = 1 to n do
                        D [ i , j ] = min (D[ i, j ], D[ i, k ] + D[ k, j ]
Return D
```

Analysis :

This algorithm takes a time of $\theta(n^3)$

EXAMPLE 2: Without Tracing the Algorithm

MVIT-CSE



A^0- Adjacency Matrix. No Intermediate vertex is allowed.

A^0	1	2	3
1	0	4	11
2	6	0	2
3	3	∞	0

A^1- Adjacency Matrix. '1' Intermediate vertex is allowed.

2 ---- 1 ---- 3 = 17>2 So Ignore
3 --- 1 ---- 2 = 7<∞ So Accept

A^1	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0

A^2- Intermediate vertex '1' and '2' is allowed.

1-----2------3=6<11 So Accept

A^2	1	2	3

MVIT-CSE

1	0	4	6
2	6	0	2
3	3	7	0

A^3- Intermediate vertex '1' , '2' and '3' are allowed.

2-----3---1= 5<6 So Accept

A^3	1	2	3
1	0	4	6
2	5	0	2
3	3	7	0

A^3 Gives all pairs of shortest path for the given graph.

3.1.4. 0/1 KNAPSACK PROBLEM

- This problem is similar to fractional knapsack problem. But the only difference is : In fractional knapsack problem we may take a fraction of an object. But in 0/1 knapsack problem either the object will be fully placed (xi=1) or the object is not placed at all. (xi =0).
- We are given ' n ' objects with weight w_i and profits p_i where i varies from 1 to n and also a knapsack with capacity ' M '.
- The problem is, we have to fill the bag with the help of ' n ' objects and the resulting profit has to be maximum.
- Formally, the problem can be stated as,

$$\text{Maximize} \sum_{i=l}^{n} x_i p_i$$

$$\text{subject to} \sum_{i=l}^{n} x_i w_i \leq M$$

- Where x_i are constraints on the solution $x_i \in \{0,1\}$. (i.e) x_i is required to be 0 or 1. if the object is selected then the unit in 1. if the object is rejected than the unit is 0. that's why it is called as 0/1, knapsack problem.
- To solve the problem by dynamic programming we form up a table T[1…n, 0…M] (ie) the size is n. where 'n' is the no. of objects and column starts with '0' to capacity (ie) 'M'.
- In the table T[i,j] will be the maximum valve of the objects i varies from 1 to n and j varies from 0 to M.

Rules To Fill The Table:-

- If i=l and j < w(i) then
  T(i,j) =0.
- If i=l and j ≥ w (i) then

MVIT-CSE

T (i,j) = p(i)

The cell is filled with the profit p[i], since only one object can be selected to the maximum.

- If i>l and j < w(i) then

  T(i,j) = T (i-l,j) .

  The cell is filled the profit of previous object since it is not possible with the current object.
- If i>l and j ≥ w(i) then

  T (i,j) = {p(i) +T(i-l,j-w(i))

  Since only 'l' unit can be selected to the maximum. It is the current profit + profit of the previous object to fill the remaining capacity of the bag.
- After the table generation, it will give the details of the profit.

Rules To Get The Combination Of Object:

- Start with the last position of i and j, T[i,j].i.e)T[n,M].

  If T[i,j] = T[i-l,j] then

  no object of 'i' is required so move up to T[i-l,j].

- After moved, Check

  if T[i,j]=T[i-l,j-w(i)]+ p[I] then

  one unit of object 'i' is selected and move up to the position T[i-l,j-w(i)]
- Repeat the same process until we reach T[i,0], then there will be nothing to fill the bag stop the process.

Example:

Capacity of the bag M=11 No.of objects n=5

W1	**W2**	**W3**	**W4**	**W5**
1	2	5	6	7
P1	**P2**	**P3**	**P4**	**P5**
1	6	18	22	28

TABLE:

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**
1	0	1	1	1	1	1	1	1	1	1	1	1
2	0	1	6	7	7	7	7	7	7	7	7	7
3	0	1	6	7	7	18	19	24	25	25	25	25
4	0	1	6	7	7	18	22	23	28	29	29	40
5	0	1	6	7	7	19	22	28	29	34	35	35

The selected objects are 3 4

Analysis:

A time Θ(nM) is necessary to construct the tble V, and thst the composition of the optimal load can be detemined in atime O(n+M).

<u>Solve problems for Practice:</u>

a. **Solve the following problem using 0/1 Knapsack problem using dynamic programming. M=10. Write the necessary rules.**

MVIT-CSE

Item	Weight	Profit
1	5	10
2	4	40
3	6	30
4	3	50

b. Solve the following 0/1 Knapsack problem using Dynamic programming. M= 15, (w1,w2,w3,w4) = (2,5,10,6) (P1,P2,P3,P4) = (50,40,60,70)

3.1.5. TRAVELLING SALESMAN PROBLEM :

- Let G(V,E) be a directed graph with edge cost $c_{ij.}$
- $c_{ij.}$ is defined such that $c_{ij} > 0$ for all i and j and $c_{ij} = \propto$,if $<i,j> \notin E$.
- Let $|V| = n$ and assume n>1.
- The traveling salesman problem is to find a tour of minimum cost.
- A tour of G is a directed cycle that include every vertex in V.
  i.e)The tour is the shortest path that starts and ends at the same vertex (ie) 1.
- The cost of the tour is the sum of cost of the edges on the tour.

Application :

1. Suppose we have to route a postal van to pick up mail from the mail boxes located at 'n' different sites.An n+1 vertex graph can be used to represent the situation.One vertex represent the post office from which the postal van starts and return.Edge <i,j> is assigned a cost equal to the distance from site 'i' to site 'j'.The route taken by the postal van is a tour and we are finding a tour of minimum length.

Every tour consists of an edge <1,k> for some $k \in V-\{\}$ and a path from vertex k to vertex 1.The path from vertex k to vertex 1 goes through each vertex in V-{1,k} exactly once.

The function which is used to find the path is

$$\mathbf{g(1,V-\{1\}) = \min\{ c_{ij} + g(j,s-\{j\})\}}$$

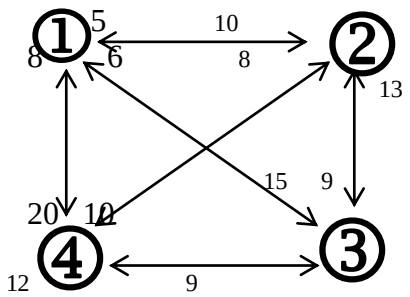
- g(i,s) be the length of a shortest path starting at vertex I going through all vertices in S and terminating at vertex 1.

$$\mathbf{g(i,s) = \min\{c_{ij} + g(j,s-\{j\})}$$

- The function g(1,v-{1}) is the length of an optimal tour.

Steps To Find The Path:

1. Find $g(i,\Phi) = c_{i1}$, 1<=i<=n, hence we can use equation(2) to obtain g(i,s) for all s to size 1.
2. That we have to start with s=1,(ie) there will be only one vertex in set 's'.
3. Then s=2, and we have to proceed until |s| <n-1.
4. For example consider the graph.



Cost Matrix

$$\begin{vmatrix} 0 & 10 & 15 & 20 \\ 5 & 0 & 9 & 10 \\ 6 & 13 & 0 & 12 \\ 8 & 8 & 9 & 0 \end{vmatrix}$$

g(i,s) $\longrightarrow$ starting position

$\downarrow$

set of nodes/vertex have to visited.

s = {1}
v ={2,3,4}
$g(i,s) = \min\{c_{ij} + g(j,s-\{j\})$

Step 1:

When s = 0 and i =1 to n.

$g(1,\Phi) = c_{11} \Rightarrow 0$
$g(2,\Phi) = c_{21} \Rightarrow 5$
$g(3,\Phi) = c_{31} \Rightarrow 6$
$g(4,\Phi) = c_{41} \Rightarrow 8$

$g(1,\{2,3,4\})=\min\{c_{12}+g(2\{3,4\}),c_{13}+g(3,\{2,4\}),c_{14}+g(4,\{2,3\})\}$
$=\min\{10+25,15+25,20+23\}$
$=\min\{35,35,43\}$
$=35$

Step2:

When $|s|$ = 1 and i =2 to 4

$g(3,\{4\}) = \min\{c_{34} + g\{4,\Phi\}\}$
$=12+8$
$=20$
$g(4,\{3\}) = \min\{c_{43} + g\{3,\Phi\}\}$
$= 9+6$
$=15$
$g(2,\{4\}) = \min\{c_{24} + g\{4,\Phi\}\} = 10+8 = 18$
$g(4,\{2\}) = \min\{c_{42} + g\{2,\Phi\}\}$
$= 8+5$
$=13$
$g(2,\{3\}) = \min\{c_{23} + g\{3,\Phi\}\}$
$=9+6$
$=15$
$g(3,\{2\}) = \min\{c_{32} + g\{2,\Phi\}\}$
$= 13+5$
$=18$

Step3:

When $|s|$ = 2 and $i \neq 1$, $1 \in s$ and $i \in s$.
$g(2,\{3,4\}) = \min\{c_{23}+g(3\{4\}),c_{24}+g(4,\{3\})\}$
$= \min\{9+20,10+15\}$

MVIT-CSE

$=\min\{29,25\}$
$=25$

$g(3,\{2,4\}) = \min\{c_{32}+g(2\{4\}),c_{34}+g(4,\{2\})\}$
$=\min\{13+18,12+13\}$
$= \min\{31,25\}$
$=25$

$g(4,\{2,3\}) = \min\{c_{42}+g(2\{3\}),c_{43}+g(3,\{2\})\}$
$=\min\{8+15,9+18\}$
$= \min\{23,27\}$
$=23$

Step4:

When $|s| = 3$

$g(1,\{2,3,4\})=\min\{c_{12}+g(2\{3,4\}),c_{13}+g(3,\{2,4\}),c_{14}+g(4,\{2,3\})\}$
$=\min\{10+25,15+25,20+23\}$
$=\min\{35,35,43\}$
$=35$

An optimal cost is 35
The shortest path is,
$g(1,\{2,3,4\}) = c_{12} + g(2,\{3,4\})$ => 1->2
$g(2,\{3,4\}) = c_{24} + g(4,\{3\})$ => 1->2->4
$g(4,\{3\}) = c_{43} + g(3\{\Phi\})$ => 1->2->4->3->1
So the optimal tour is **1 → 2 → 4 →3 → 1**

Analysis:

- An algorithm that proceeds to find an optimal tour by using above 2 equations require $(n^2 2^n)$ time.
- Space needed by this algorithm is $O(n.2^n)$.This is too large even for modest values of n. This is a one main drawback of this problem.

EXAMPLE 2:

	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	7	13	0	12
4	7	6	9	0

g(i,s) → starting position

set of nodes/vertex have to visited.

s = {1}
v ={2,3,4}
$g(i,s) =\min\{c_{ij} +g(j,s-\{j\})$

Step 1:

When s = 0 and i =1 to n.

$g(1,\Phi,1) = c_{11}$ => 0
$g(2,\Phi,1) = c_{21}$ => 5
$g(3,\Phi,1) = c_{31}$ => 7
$g(4,\Phi,1) = c_{41}$ => 7

MVIT-CSE

$g(1,\{2,3,4\},1)=\min\{c_{12}+g(2\{3,4\},1),c_{13}+g(3,\{2,4\},1),c_{14}+g(4,\{2,3\},1)\}$
$=\min\{10+26,15+23,20+22\}$
$=\min\{36,38,42\}$
$=36$

Step2:

When $|s|=1$ and $i=2$ to 4

$g(2,\{3\},1) = \min\{c_{23} +g\{3,\Phi,1\}\}$
$=9+7$
$=16$

$g(2,\{4\},1) = \min\{c_{24} +g\{4,\Phi,1\}\}$
$= 10+7$
$=17$

$g(3,\{2\},1) = \min\{c_{32} +g\{2,\Phi,1\}\}$
$= 13+5$
$=18$

$g(3,\{4\},1) = \min\{c_{34} +g\{4,\Phi,1\}\}$
$=12+7$
$=19$

$g(4,\{2\},1) = \min\{c_{42} +g\{2,\Phi,1\}\}$
$= 6+5$
$=11$

$g(4,\{3\},1) = \min\{c_{43} +g\{3,\Phi,1\}\}$
$= 9+7$
$=16$

Step3:

When $|s| = 2$ and $i \neq 1$, $1 \in s$ and $i \in s$.

$g(2,\{3,4\},1) = \min\{c_{23}+g(3\{4\},1),c_{24}+g(4,\{3\},1)\}$
$= \min\{9+19,10+16\}$
$=\min\{28,26\}$
$=26$

$g(3,\{2,4\},1) = \min\{c_{32}+g(2\{4\},1),c_{34}+g(4,\{2\},1)\}$
$=\min\{13+17,12+11\}$
$= \min\{30,23\}$
$=23$

$g(4,\{2,3\},1) = \min\{c_{42}+g(2\{3\},1),c_{43}+g(3,\{2\},1)\}$
$=\min\{6+16,9+18\}$
$= \min\{22,27\}$
$=22$

Step4:

When $|s| = 3$

$g(1,\{2,3,4\},1)=\min\{c_{12}+g(2\{3,4\},1),c_{13}+g(3,\{2,4\},1),c_{14}+g(4,\{2,3\},1)\}$
$=\min\{10+26,15+23,20+22\}$
$=\min\{36,38,42\}$

MVIT-CSE

=36

An optimal cost is 36

The shortest path is,

$g(1,\{2,3,4\}) = c_{12} + g(2,\{3,4\})$ => 1->2

$g(2,\{3,4\}) = c_{24} + g(4,\{3\})$ => 1->2->4

$g(4,\{3\}) = c_{43} + g(3\{\Phi\})$ => 1->2->4->3->1

So the optimal tour is **1 → 2 → 4 →3 → 1**

<u>Solve problems for Practice:</u>

1. **Find the Shortest tour for TSP using dynamic programming for the following graph.**

	a	b	c	d
a	0	2	9	10
b	1	0	6	4
c	15	7	0	8
d	6	3	12	0

3.1.6. CHAINED MATRIX MULTIPLICATION:

If we have a matrix A of size p×q and B matrix of size q×r. The product of these two matrix C is given by,

$$\mathbf{c_{ij} = \Sigma\, a_{ik}\, b_{kj}\ ,\ 1\le i \le p,\ 1\le j \le r\ ,\ 1\le k \le q.}$$

It requires a total of **pqr** scalar multiplications.

Matrix multiplication is associative.i.e)(AxB).C=A.(BxC). So if we want to calculate the product of more then 2 matrixes m= m1m2……. m_n. For example, multiplication of 4 matrices A x B x C x D.The order of the matrices are given below.

A = 13 × 5

B = 5 × 89

C = 89 × 3

D = 3 × 34

We can multiply these 4 matrices in any no. of the sequences. Now, we will see some of the sequences:

1st Sequence: M = (((A.B).C).D)

A.B C

= (13 * 5 * 89) * (89 * 3)

A.B.C. D

= (13 * 89 * 3) * (3 * 34)

A.B.C.D

= 13 * 3 * 34

No.of multiplications required = 13 * 5 * 89 + 13 * 89 * 3 + 13 * 3 * 34

= 10,582

2nd Sequence: M = (A * B) * (C * D)

= 13 * 5 * 89 + 89 * 3 * 34 + 13 * 89 * 34

= 54201 no. of multiplications

3rd Sequence: M = (A.(BC)) . D

= 5 * 89 * 3 + 13 * 5 * 3 + 13 * 3 *34

= 2856 no. of multiplications

For comparing all these sequences, (A(BC)).D sequence requires less no. of multiplications. For finding the no. of multiplications we going for Dynamic programming method.

MVIT-CSE

STRAIGHT FORWARD METHOD:

Our aim is to find the total no. of scalar multiplications required to compute the product of n matrices. Let M= (M1,M2,……Mn) be the chain of matrix to be calculated using the dynamic programming method. First simply parenthesize the expression in every possible fashion. Then count each time how many multiplications are required.

- To find the product of n matrices:
  - M= (M1,M2,……Mn).
- Parenthesize the product of n matrices.i.e) Split it in-between ith and i+1th matrix.
  - M=(M1,M2,……Mi) and (Mi+1,Mi+2,……Mn)

- In dynamic programming we always start with the smallest instances and continue till we reach the required size.
- To compute the matrix product in optimal way using dynamic programming method, we construct a table m_{ij}, $1 \le i \le j \le n$, Where m_{ij} gives the optimal solution.
- Sizes of all the matrixes are stored in the array d[0..n]
- Build the table diagonal by diagonal;
- Diagonal s contains the elements m_{ij} such that j-1 =s.

Rules To Fill The Table Mij:

s =0,1,……n-1

- If s=0 then

  m(i,i) =0 for i =1,2,……n
- If s=1 then

  m(i,i+1) = d(i-1)*di *d(i+1) for i=1,2,……n-1.

- If 1< s <n then

  $m(i,i+s) = \min(m_{ik}+m_{k+1,i+s}+d_{i-1}d_kd_{i+s})$ for $i \le k \le i+s$ and i = 1,2,……n-s

Example:

A=>13×5
B=>5×89
C=>89×3
D=>3×34

Single dimension array is used to store the sizes.

d[0]=13
d[1]=5
d[2]=89
d[3]=3
d[4]=34

If s=0 then

m(1,1)=0
m(2,2)=0
m(3,3)=0
m(4,4)=0

If s=1 then mi,i+1 = d(i-1)*di *d(i+1)

for i= 1 to 3

when i=1 m12 = d0*d1*d2 =13*5*89=5785
when i=2 m23 = d1*d2*d3 =5*89*3=1335
when i=3 m34 = d2*d3*d4 =89*3*34=9078

MVIT-CSE

if s=2 then $m_{i,i+s} = \min(m_{ik}+m_{k+1,i+s}+d_{i-1}d_kd_{i+s})$

for i= 1 to 2

when i=1 for k=1 to 2

when i=1,k=1

$m_{13} = \min(m_{11}+m_{23}+d_0d_1d_3 , m_{12}+m_{33}+d_0d_2d_3)$

=(0+1335+(13*5*3),5785+0+(13*89*3))

=min(1530,9256)

=1530

when i=1,k=2

$m_{13} = \min(m_{22}+m_{34}+d_1d_2d_4 , m_{23}+m_{44}+d_1d_3d_4)$

=(0+9078+(5*89*34),1335+0+(5*3*34))

=min(24208,1845)

=1845

When i=2 for k= 2 to 3

When i=2 ,k=2

m24 = m22+m34+d1d2d4

=0+9048+5 x 89 x34

=24208

When i=2 ,k=3

m24 = m23+m44+d1d3d4

=1335+0+5 x 3 34

=1845

m24 =min(24208,1845)

=1845

if s=3 then $m(i,i+s) = \min(m_{ik}+m_{k+1,i+s}+d_{i-1}d_kd_{i+s})$ for $i \le k \le i+s$ and $i = 1,2,\ldots\ldots n-s$

i=1 to 4-3 and k= 1 to 3

k=1 k=2 k=3

$m_{14} = \min(m_{11}+m_{24}+d_0d_1d_4 , m_{12}+m_{34}+d_0d_2d_4, m_{13}+m_{44}+d_0d_3d_4)$

=min(4055,54201,2856)

=2856

The matrix table m_{ij} will look like,

	1	2	3	4	
1	0	5785	1530	2856	s=3
2		0	1335	1845	s=2
3		0	9078	s=1	
4			0	s=0	

Here there is no need to fill the lower diagonal. The solution to the original problem is given by m_{1n} in the table. The execution time of the algorithm is $\Theta(n^3)$.

MVIT-CSE

3.1.7. APPROACHES USING RECURSION

Although dynamic programming algorithms give efficient result, there is some unsatisfactory about the bottom up approach.B'coz it leads to compute values that might be completely irrelevant. SO we go for the top down approach to achieve the same efficiency.

The matrix multiplication using dynamic programming can be replaced that means replacing the table M by a function fm which is calculated as required in other words we would like to find a function fm such that fm(I,j) equal to mi,j for 1<=I<=j<=n;But that can be calculated recursively, writing such a function is simple.We have to program the rules for calculating M.

```
Function fm(i,j)
{
        if i=j then {only one matrix is involved}
            return 0;
        m:=infinity
        for k:= i to j-1 do
                m=min   (m,fm(i,k)+fm( k+1,j)+d[i-1]d[k]d[j]);
        return m;
 }
```

To find the number of scalar multiplication to calculate m1.m2.....mn we simply call fm(i,n).

3.1.8. MEMORY FUNCTIONS:

Some times simple recursive formulation of solution leads to an in efficient program as common sub instances or solved independently more than one's.

Dynamic programming allows us to avoid this at cost of complicating algorithm. But in this technique it also allows us to solve some irrevalent sub instances. So we combine the advantages of both the technique and retain the simplicity of recursive formulation without losing the efficiency offered by dynamic (ie) we are using memory function.

To the recursive program we add a table of a necessary size. Initially all the entries in the table hold a special value to show they have not yet be calculated .There after whenever we call the function we first look in the table to see whether it has been already evaluated with same set of parameter,if so we return the value in that table if not we go ahead and calculated the value,we save it at appropriate table. In this way it never necessary to calculate the function twice for the same value of parameter.

```
Function fm_mem(i,j)
{
    if (i=j)then return 0;
      if mtab[i,j]>=0 then return mtab[i,j];
      m :=infinity;
      for k:= i to j-1 do
      m := min(m,fm_mem(i,k)+fm_mem(k+1,j)+d[i-1]d[k]d[j]);
      mtab[i,j]:=m;
      return m;
}
```

3.2. BASIC SEARCH AND TRAVERSAL TECHNIQUE (FOR FURTHER NOTES REFER DATA STRUCTURES)

MVIT-CSE

3.2.1. THE TECHNIQUE

It is divided into 2 categories. They are,

1.Traversal Method 2.Search Method

Traversal Method:

- Traversal means visiting all the nodes of the graph
- Applicable only to binary trees.
- It examines every node in the given data object instance.

Search Method:

- In search method all the nodes of the graph are visited, until the searching node is found.
- Applicable to graphs.
- Not examine all the vertices.
- If any node is examined, then it is said to be visited.

3.2.1.1. TECHNIQUES FOR BINARY TREES:

To solve many problems, it needs manipulation of binary trees/ graphs/ trees. This manipulation requires to determine a vertex/subset of vertices that satisfies a given problem.

Example 1:

Let B be a one binary tree. It consists of many nodes. Now our objective is to find all the vericves in the binary tree with a data value less than x.

Example 2:

To find all the vertices in a graph G that can be reached from another given vertex v.

The solution to these problems can be found by examining all the vertices in graph/tree. This takes the form of search a data in the data object. When the search involves the examination of every vertex in the object being searched, it is called as a traversal.

Operations Performed on a Binary Tree:

- Traversing a tree => Visiting each node in the tree exactly once.

When traversing a tree ,we can visit the nodes in the following orders.

Pre Order - DLR
Post Order - LRD
In order - LDR

INORDER:

First visit the left node of the root node.Then visit the root node and right node of the root node.Do this process continuously till we traverse all the nodes in the tree.

Recursive formulation of inorder traversal

```
treenode=record
{
        Type data;//Type ios the data type of data.
        Treenode *lchild;Treenode *rchild;
}
Algorithm InOrder(t)
//t is a binary tree.Each of t has three fields:lchild,data and rchild.
{
        if t ≠ 0 then
        {
                InOrder(t->lchild);
                Visit(t);
                InOrder(t->rchild);
        }
}
Algorithm PreOrder(t)
```


MVIT CSE

```
//t is a binary tree.Each of t has three fields:lchild,data and rchild.
{
        if t ≠ 0 then
        {
                Visit(t);
                PreOrder(t->lchild);
                PreOrder(t->rchild);
        }
}
Algorithm PostOrder(t)
//t is a binary tree.Each of t has three fields:lchild,data and rchild.
{
        if t ≠ 0 then
        {
                PostOrder(t->lchild);
                PostOrder(t->rchild);
                Visit(t);
        }
}
```

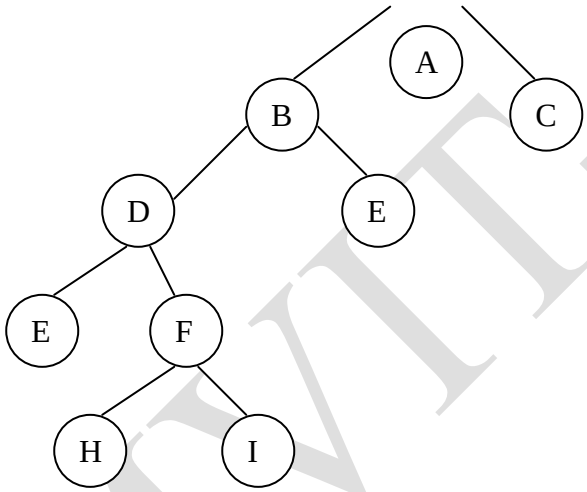


Figure 3.2.1.

The above picture shows a binary tree.Visit(t) – Visiting a node requires only the printing of its data field.The output resulting from inorder,preorder and postorder traversal is FDHGIBEAC, ABDFGHIEC and FHIGDEBCA respectively.

Analysis:

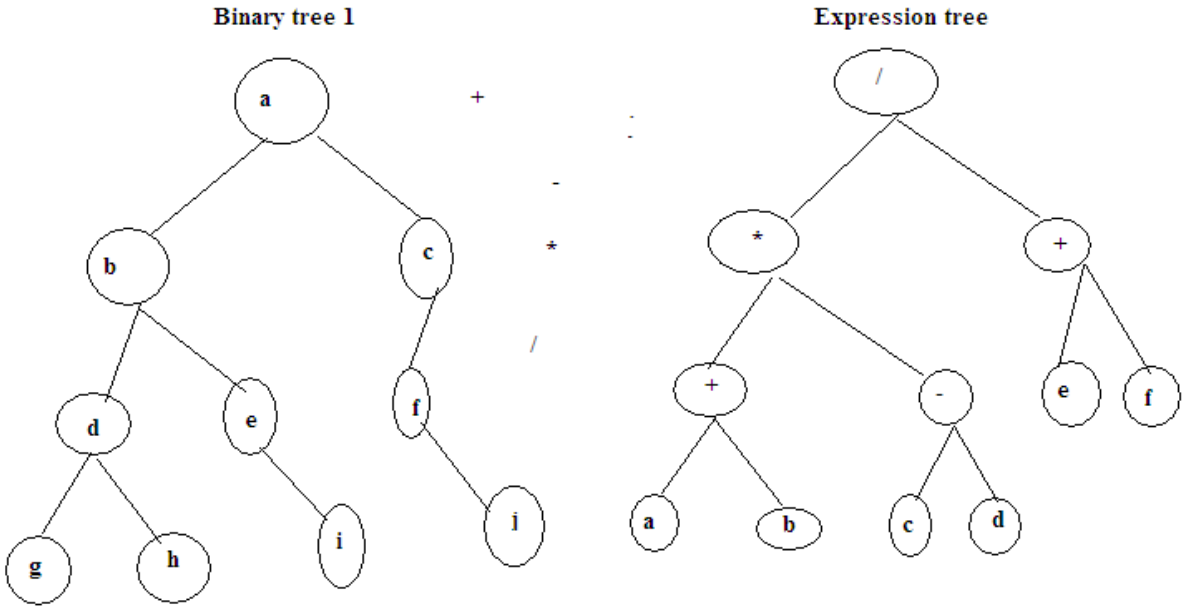
The total time taken by the traversal is θ(n).the additional space needed is that for the recursion stack.If t has depth d,then this space is θ(d).For an n-node binary tree,d≤n and so S(n)=O(n).

Solve problems for practice:

1. **Illustrate the following binary trees and their traversal in preorder , in order, and postorder.**

MVIT-CSE

a.



Binary Tree:

a. **Inorder : gdhbeiafjc**
b. **Preorder: abdgheicfj**
c. **Postorder: ghdiebjfca**

Expression Tree:

a. **Inorder : a+b*c-d/e+f**
b. **Preorder: /*+ab-cd+ef**
c. **Postorder: ab+cd-*ef+/**

3.2.1.2. TECHNIQUES FOR GRAPHS:

- The fundamental problem concerning graphs is the reachability problem.
- In it simplest from it requires us to determine whether there exist a path in the given graph, G =(V,E) such that this path starts at vertex ‘v’ and ends at vertex ‘u’.
- A more general form is to determine for a given starting vertex v €V all vertices u such that there is a path from v to u.
- This problem can be solved by starting at vertex ‘v’ and systematically searching the graph ‘G’ for vertex that can be reached from ‘v’.
- We describe 2 search methods for this.
    - Breadth first Search and Traversal.
    - Depth first Search and Traversal.

3.2.2. BREADTH FIRST SEARCH AND TRAVERSAL:

- In Breadth First Search we start at a vertex ‘v’ and mark it as having been reached (visited).
- The vertex ‘v’ is at this time said to be unexplored.
- A vertex is said to have been explored by an algorithm when the algorithm has visited all vertices adjust from it.

- All unvisited vertices adjacent from ‘v’ are visited next. These are new unexplored vertices.
- Vertex ‘v’ has now been explored. The newly visit vertices haven’t been explored and are put on the end of a list of unexplored vertices.
- The first vertex on this list in the next to be explored. Exploration continues until no unexplored vertex is left.
- The list of unexplored vertices operates as a queue and can be represented using any of the start queue representation.

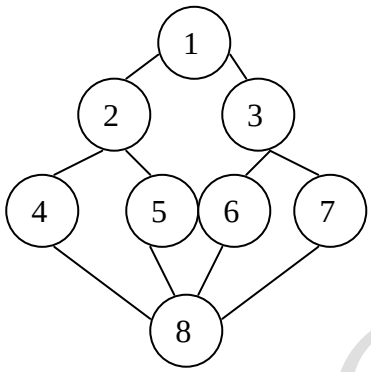


Figure 3.2.2.

Psuedo Code Algorithm for breadth first search:

```
Algorithm BFS (v)
// A breadth first search of ‘G’ is carried out.
// beginning at vertex-v; For any node i, visited[i]=1..
// if ‘i’ has already been visited. The graph ‘G’
// and array visited [] are global; visited []
// initialized to zero.
{
        u=v; // q is a queue of unexplored vertices
        visited (v)= 1;
        repeat
        {
            for all vertices ‘w’ adjacent from u do
            {
                if (visited[w]=0) then
                {
                    Add w to q;
                      visited[w]=1
                      }
            }
            if q is empty then return;// No unexplored vertex,delete u from q;
            //get first unexplored vertex
        } until (false)//q becomes empty
}
```

Tracing of BFS Algorithm:

```
        v=1 u=1     q=1     visited[1]=1
u=1     for w= 2 to 3
        when w=2
```


MVIT-CSE

```
                if visited[2]=0 then true     q=1,2     visited[2]=1
            when w=3
                if visited[3]=0 then true     q=1,2,3  visited[3]=1
            If q is not empty delete 1 from q. So q=2,3
u=2         for w= 4 to 5
            when w=4
                if visited[4]=0 then true     q=2,3,4  visited[4]=1
            when w=5
                if visited[5]=0 then true     q=2,3,4,5      visited[5]=1
            If q is not empty delete 2 from q. So q=3,4,5
u=3         for w= 6 to 7
            when w=6
                if visited[6]=0 then true     q=3,4,5,6      visited[6]=1
            when w=7
                if visited[7]=0 then true     q=3,4,5,6,7   visited[7]=1
            If q is not empty delete 3 from q. So q=4,5,6,7
u=4         for w= 8
            when w=8
                if visited[8]=0 then true     q=4,5,6,7,8   visited[8]=1
            If q is not empty delete 4 from q. So q=5,6,7,8
u=5         for w= 8
            when w=8
                if visited[8]=0 then false
            If q is not empty delete 5 from q. So q=6,7,8
u=6         for w= 8
            when w=8
                if visited[8]=0 then false
            If q is not empty delete 6 from q. So q=7,8
u=7         for w= 8
            when w=8
                if visited[8]=0 then false
            If q is not empty delete 7 from q. So q=8
u=8         for w= 8
            when w=8
                if visited[8]=0 then false
            If q is not empty delete 7 from q. So q=Nil
```

A breadth first search of the graph of figure 3.2.2. starting at vertex 1 results in the vertices being visited in the order 1,2,3,4,5,6,3,7,8.

Psuedo Code Algorithm for Breadth First graph Traversal:

```
Algorithm  BFT(G,n)
{
    for i= 1 to n do
       //mark all vertices unvisited
       visited[i] =0;
    for i =1 to n do
                    if (visited[i]=0)then BFS(i);
}
```

MVIT-CSE

Here the time and space required by BFT on an n-vertex, e-edge graph one O(n+e) and O(n) resp if adjacency list is used. if adjacency matrix is used then the bounds are $O(n^2)$ and O(n) respectively.

3.2.3. DEPTH FIRST SEARCH AND TRAVERSAL:

A depth first search of a graph differs from a breadth first search in that the exploration of a vertex v is suspended as soon as a new vertex is reached. At this time the exploration of the new vertex u begins. When this new vertex has been explored, the exploration of u continues. The search terminates when all reached vertices have been fully explored. This search process is best-described recursively.

A depth first search of the graph of figure 3.2.2. starting at vertex 1 results in the vertices being visited in the order 1,2,4,8,5,6,3,7.

A depth first traversal of a grpah is carried out by repeatedly calling DFS ,with anew unvisited starting each time.The algorithm for this (DFT) differs from BFT only in that the call to BFS(i) is replaced by a call to DFS(i).

Psuedo Code Algorithm for Depth First Search of a graph:

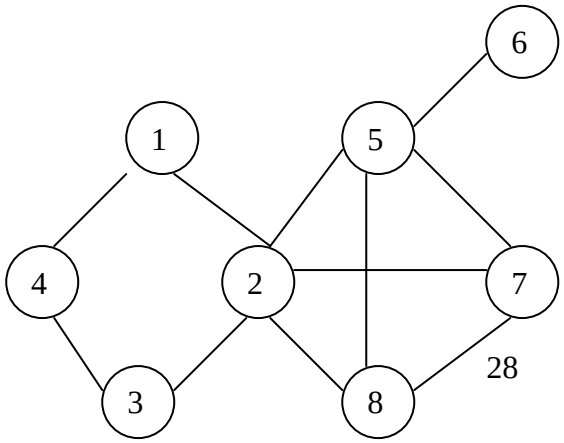
```
Algorithm DFS(V)
//given an undirected (directed graph)
//G=(V,E) with n vertices and an array visited[]
//initially set to zero, this algorithm of visits all the vertices reachable
//from v.
//G and visited[] are global
{
    visited[v]:=1;
        for each vertex w adjacent from v do
            {
                if(visited[w]=0) then DFS(w);
            }
}
```

BFS & DFS are 2 fundamentally different search methods. In BFS, a node is fully explored before the exploration of any other node begins. The next node to explore is the first unexplored node remaining. In DFS the exploration of a node is suspended as soon as a new unexplored node is reached. T he exploration of this new node is immediately begun.

3.2.4. BICONNECTED COMPONENTS :

Articulation Point:

A vertex ‘v’ in a connected graph ‘G’ is an articulation point, iff the deletion of vertex v together with all vertices incident to v disconnects the graph into 2 or more non empty components.



28

MVIT-CSE

Figure 3.2.3 – Graph G

Biconnected Graph:

A graph 'G' is biconnected, iff it contains no articulation points.

Real Time Example:

'G' is a graph representing a communication network, vertices represent communication station, edges represent communication lines.

(i) if communication station has articulation point :

→ if any communication station 'i' is failed , then we cannot communicate with another station.

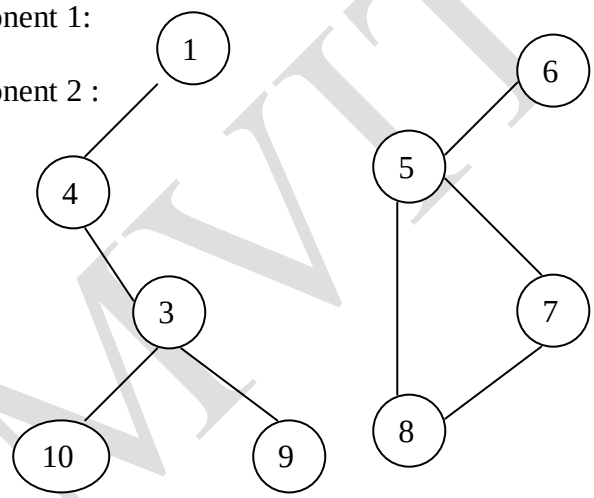
(ii) if the graph has no articulation point:

→if any station 'i' is failed, then we can communicate with another station, other than 'i'.

In the above graph G ,if we delete the vertex 2 ,then it creates 2 non empty components.

Component 1:

Component 2 :



If we delete any other vertex other than 2,3, and 5 ,then we get only one component exactly. So the above graph is not biconnected.

MVIT-CSE

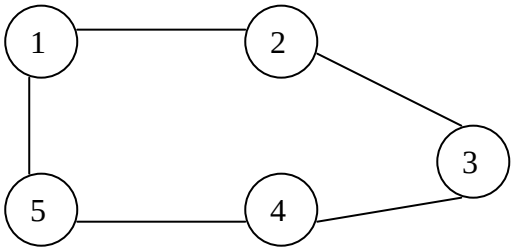


Figure 3.2.4

The figure 3.2.4. represents the graph which is biconnected.B'coz, it has no articulation points.

If a graph is not biconnected ,then we may include set of edges that makes the graph biconnected .we can determine that edge if we know the maximal subgraphs of G that are biconnected.

Let G(V,E) be a graph that is not biconnected. To make it biconnected , first identify the articulation points by the algorithm. Once it has been determined that a connected graph G is not biconnected, it may be desirable to determine the set of edges whose inclusion makes the graph biconnected. Determining such a set of edges is facilitated if we know the maximal subgraphs of G that are biconnected.G'(V',E') is a maximal biconnected subgraph of G iff G has no biconnected subgraph G"(V",E'') such that V' V'' and E' E''.A maximal biconnected subgraph is a biconnected component.

Biconnected components should contain atleast 2 vertices.Biconnected component itself has no articulation point.The graph G can be transformed into a biconnected graph by using the edge addition scheme of Algorithm to constuct a biconnected graph. Note that once the edges (v_i,v_{i+1}) of line 6 are added, vertex a is no longer an articulation point. Hence following the addition of edges corresponding to all articulation points,G has no articulation points and so it is biconnected. If G has p articulation points and b biconnected components, the scheme of algorithm introduces exactly b-p new edges into G.

Algorithm To Construct A Biconnected Graph:

```
1   for each articulation point 'a' do
2   {
3           let B1,B2……..Bk be the biconnected components containing vertex a;
4          let Vi,Vi !=a be a vertex in Bi, 1<=i<=n;
5         add to 'G' the edges (Vi,Vi+1),1<=i<=n;
6   }
```

The biconnected components of the graph of figure 3.2.3 are shown below.figure 3.2.5

MVIT-CSE

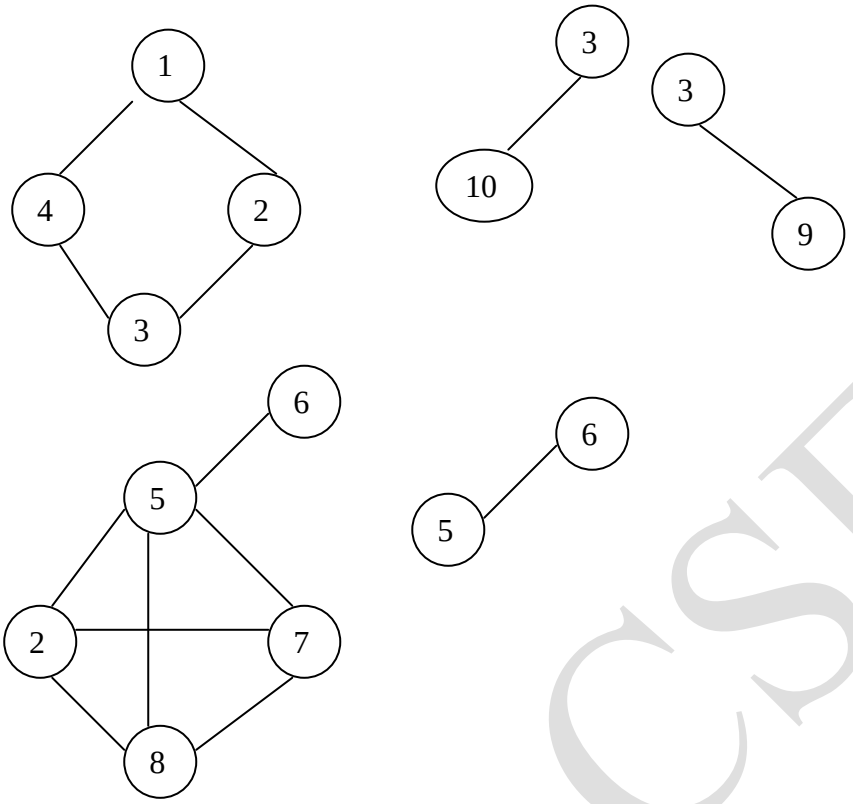


Figure 3.2.5. Biconnected components of graph of figure 3.2.4.

Pseudo Code To Determine Bicomponents :

```
Algorithm Bicomp (u,v)
{
    //u is the start vertex for depth first search.
    // v is its parents ,if any in the depth first spanning tree .
    //it is assumed that ,the global array 'dfn' is initially zero and that the global variable
    // 'num' is initialized to 1.
    //'n' is the number of vertices
    {
        dfn[u]:=num;L[u]:=num;num=num+1;
            for each vertex 'a' adjacent from 'u' do
                {
                    if((v!=w)&&(dfn[w]<dfn[u])) then
                        add [u,w] to the top of the stack S;
                    if(dfn[w]=0) then
                        {
                            if(L[w]>=dfn[u]) then
                              {
                                  write("New Bicomponents");
                                repeat
                                 {
                                     delete  an edge from top of stack s;
```


MVIT-CSE

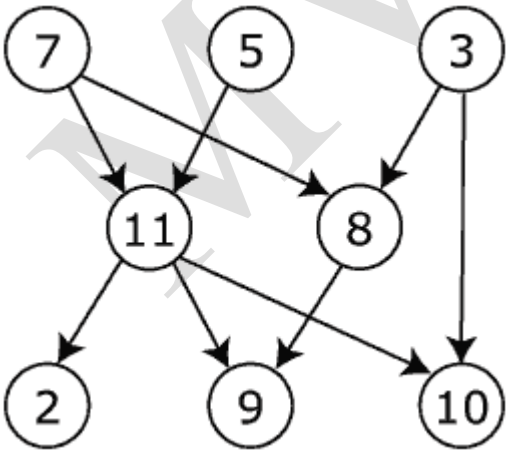
```
                    let the edge be (x,y);
                     write(x,y);
                }
                until(((x,y)=(u,w))||((x,y)=(w,u)));
            }
            Bicomp(w,u);//w is unvisited
             L(u):=min(L(u),L(w));
        }
        else if(w!=v) then
        L(u):=min(L(u),dfn[w]);
    }
}
```

3.2.5. TOPOLOGICAL SORTING

In graph theory, a topological sort or topological ordering of a directed acyclic graph (DAG) is a linear ordering of its nodes which is compatible with the partial order R induced on the nodes where x comes before y (xRy) if there's a directed path from x to y in the DAG. An equivalent definition is that each node comes before all nodes to which it has outbound edges. Every DAG has one or more topological sorts.

Example:

The canonical application of topological sorting (topological order) is in scheduling a sequence of jobs. The jobs are represented by vertices, and there is an edge from x to y if job x must be completed before job y can be done (for example, washing machine must finish before we put the clothes to dry). Then, a topological sort gives an order in which to perform the jobs. This has applications in computer science, such as in instruction scheduling, ordering of formula cell evaluation in spreadsheets, dependencies in makefiles, and symbol dependencies in linkers.



The graph shown above has many valid topological sorts, including:

- 7,5,3,11,8,2,10,9
- 7,5,11,2,3,10,8,9
- 3,7,8,5,11,10,9,2

MVJ-CSE

- 3,5,7,11,10,2,8,9

In this problem, the functions are:

i) Decide whether a vertex has any predecessor.
ii) Decide a vertex together with all its incident edges.

- This can be efficiently done if for each vertex a count of the no of its immediate predecessor is kept.
- This can be easily implemented if the network is represented by its adjacency lists.

The following algorithm assumes that the network is represented by adjacency lists.

ALGORITHM:

```
Procedure Topological_order (count,vertex,link,n)
//the n vertices of an AOV-network are listed in topological order .
the network is represented as a set of adjacency lists with
COUNT(i)=the in-degree of vertex i//
top← 0     //initialize stack//
for i ← 1 to n do    //create a linked stack of vertices with      no predecessors //
    if COUNT (I)=0 then [ COUNT(i) ← top;top←I]
    end
    for i ←1 to n do //print the vertices in topological order //
    if top = 0 then [print ('network has a cycle ');stop]
    j ← top;top←COUNT (top);print(j)//unstack a vertex//
    ptr ←LINK(j)
    while ptr   <> 0 do
    //decrease the count of successor vertices of j//
    K← VERTEX(ptr)  //  K is a successor of j//
    COUNT(K) ← COUNT(K)-1  //  Decrease count//
    If COUNT(K)=0 //  Add vertex K to stack//
    Then[COUNT(K) ← top; top←K]
    Ptr ←LINK(ptr)
    End
    End
    End TOPOLOGICAL_ORDER.
```

COUNT LINK VERTEX LINK

0	
1	
1	
1	
3	0
2	0

The head nodes of these lists contain two fields : COUNT & LINK. The COUNT field contains the in-degree of that vertex and LINK is a pointer to the first node on the adjacency list. Each list node has 2 fields:VERTEX & LINK.

COUNT fields can be set at the time of i/p. When edge < i ,j > is i/p the count of vertex j is incremented by 1. The list of vertices with zero count is maintained as a stack.

MVIT-CSE

MVIT-CSE